Nombre: Morales Gonzales Emmanuel – **Boleta**: 2025630602, Romero Sayún Jonathan - **Boleta**: 2025630298 ° Grupo: 3CV4

Ejercicio 2, Parte A — El sistema gestor en un contenedor

1. Contenedor frente a máquina virtual

Un contenedor es un proceso (o conjunto de procesos) del sistema anfitrión que se ejecuta aislado del resto mediante mecanismos del propio núcleo de Linux —espacios de nombres (namespaces) y grupos de control (cgroups)—, con su propio sistema de archivos, su propia vista de la red y sus propios límites de CPU y memoria. Una máquina virtual, en cambio, emula hardware completo mediante un hipervisor y ejecuta encima un sistema operativo huésped íntegro, con su propio núcleo.

De esa diferencia estructural se desprenden las tres comparaciones pedidas:

- **Arranque**. El contenedor no arranca un núcleo ni inicializa servicios de sistema: sólo lanza un proceso. Arranca en el orden de cientos de milisegundos a pocos segundos. La máquina virtual debe completar el arranque de un sistema operativo entero, lo que toma decenas de segundos o minutos.
- **Tamaño**. La imagen de un contenedor contiene únicamente las bibliotecas y binarios que la aplicación necesita: típicamente decenas o pocos cientos de megabytes (postgres:16 ronda los 400 MB, y variantes alpine bastante menos). Una imagen de máquina virtual incluye el sistema operativo completo y se mide en gigabytes.
- **Aislamiento**. La máquina virtual ofrece un aislamiento más fuerte, porque la frontera es el hipervisor y cada huésped tiene su propio núcleo. Los contenedores comparten el núcleo del anfitrión: el aislamiento es lógico, no de hardware, y una vulnerabilidad de escape del núcleo compromete a todos. A cambio, el contenedor es mucho más ligero y reproducible.

2. Definiciones

- **Imagen**. Plantilla inmutable, de sólo lectura y organizada en capas, que contiene el sistema de archivos y la configuración necesarios para ejecutar una aplicación. Se identifica por nombre y etiqueta: postgres:16. Una imagen no se ejecuta: se instancia.
- **Contenedor**. Instancia en ejecución de una imagen. Añade sobre las capas de sólo lectura una capa de escritura efímera. Es desechable: puede detenerse, eliminarse y volver a crearse desde la misma imagen.
- **Volumen**. Área de almacenamiento persistente gestionada por Docker, con ciclo de vida independiente del contenedor, que se monta en una ruta del sistema de archivos del contenedor. En pgdata:/var/lib/postgresql/data, el volumen pgdata cubre el directorio donde PostgreSQL escribe sus datos.
- **Puerto publicado**. Asignación que expone un puerto interno del contenedor en una interfaz del anfitrión, permitiendo que clientes externos se conecten. En -p 5432:5432, el primer

número es el puerto del anfitrión y el segundo el del contenedor; sin esta publicación, el servidor PostgreSQL escucharía sólo dentro de la red del contenedor y psql desde el sistema operativo no lo alcanzaría.

3. Por qué el volumen es indispensable

La capa de escritura de un contenedor forma parte del contenedor mismo, no de la imagen. Al eliminar el contenedor (docker rm, o docker compose down), esa capa se destruye junto con él.

Si no se declara un volumen, PostgreSQL escribe su directorio de datos (/var/lib/postgresql/data) en esa capa efímera. El comportamiento observable es el siguiente: mientras el contenedor exista, todo funciona con normalidad — las bases de datos, las tablas y las filas persisten a través de reinicios con docker stop y docker start, porque detener no es eliminar—. Pero en el momento en que el contenedor se elimina y se vuelve a crear a partir de la imagen, el nuevo contenedor arranca con un directorio de datos vacío: PostgreSQL ejecuta su inicialización desde cero y la base de datos practica1, sus tablas y sus registros han desaparecido de forma irrecuperable. No hay advertencia ni error; simplemente ya no están.

Existe además una razón secundaria de rendimiento: el controlador de almacenamiento por capas (overlay) es ineficiente para la escritura intensiva y aleatoria que hace un gestor de bases de datos. El volumen escribe directamente sobre el sistema de archivos del anfitrión y evita esa penalización.

Por eso, para cualquier servicio con estado —y un SGBD es el ejemplo canónico— declarar el volumen no es una optimización, sino un requisito de corrección.

Referencias

Docker, Inc. (2024). Docker Docs: Volumes. https://docs.docker.com/storage/volumes/

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. Linux Journal, 2014(239), 2. PostgreSQL Global Development Group. (2024). PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/